

Implementation of an Offline-First Progressive Web App Architecture for Transactional System Resilience

Nugraha^{1)*}, Somantri²⁾, Ahmad Rofiqi³⁾

¹⁾²⁾³⁾Informatics Engineering Study Program, Faculty of Engineering, Computer, and Design, Nusa Putra University, Sukabumi, Indonesian

¹⁾nugraha@nusaputra.ac.id

²⁾somantri@nusaputra.ac.id

³⁾ahmad.rofiqi_ti22@nusaputra.ac.id

Article history:

Received 02 Jul 2026;
Revised 21 Jul 2026;
Accepted 05 Aug 2026;
Available online 10 August 2026

Keywords:

IndexedDB
Offline-First
Point of Sale
Progressive Web App
Service Worker

Abstract

Transactional Point of Sale (POS) systems require continuous transaction capture, responsive user interaction, and low tolerance for queue loss under unstable connectivity. The Mangkasir web dashboard, developed using Next.js, previously followed an online-only workflow in which failed HTTP requests during network disruption could interrupt cashier operations and leave transaction payloads unsaved. This study aims to evaluate the architectural feasibility of an Offline-First Progressive Web App (PWA) strategy for preserving write-heavy POS transaction queues during controlled network failure scenarios. Using a prototyping development model, the architecture integrates Service Worker-based request interception, IndexedDB-based local queue persistence, and Background Sync-based asynchronous transmission after reconnection. The implementation was evaluated through Black Box Testing, load synchronization testing, and controlled multi-device concurrency testing. The results show that the proposed architecture successfully intercepted failed transaction requests, maintained interface responsiveness, and preserved 100% of locally queued transactions under the tested conditions. After connectivity was restored, the Background Sync mechanism synchronized queued payloads asynchronously, processing 100 transactions in 42.60 seconds without observed transaction loss in the evaluated scenarios. Multi-device testing further indicated that simultaneous offline queues from three devices were synchronized without detected duplication or race-condition errors. These findings support the feasibility of Offline-First PWA architecture for improving transactional resilience, but they should not be interpreted as evidence of complete operational readiness. Further validation is required for payment reconciliation, inventory consistency, server failure recovery, browser compatibility, interrupted synchronization, transaction cancellation, and the security of locally stored financial payloads.

I. INTRODUCTION

The development of information technology has transformed web applications from static information providers into operational platforms for dynamic and time-sensitive business processes. Contemporary web systems must manage interactions among users, devices, databases, and servers, especially in sectors that rely on real-time transaction processing. Transactional systems are central because they record, validate, and execute financial data mutations. Transaction Processing Systems provide structured operational records and support managerial decision-making [1]. In cloud-based environments, architecture must consider scalability, latency, operational cost, and reliability because online transaction processing depends on continuous interaction between clients, servers, and databases [2]. A Point of Sale (POS) system not only records sales but also affects inventory updates, payment confirmation, cashier accountability, and managerial reporting. Integrated POS platforms connect sales processing with inventory control and broader business operations [3]. In this article, transactional

* Corresponding author

resilience is defined within technical boundaries as the capability of a POS application to preserve submitted transaction queues during connection loss, maintain user-interface responsiveness, synchronize locally stored payloads after reconnection, and prevent duplicate transaction entries under controlled synchronization scenarios. It does not imply complete end-to-end business correctness across payment settlement, inventory reconciliation, fiscal auditing, or server-side disaster recovery.

Because of this role, transactional systems are mission-critical applications that require high availability and low tolerance for failure or data loss. In retail operations, transaction failure may disrupt sales records, stock accuracy, payment reconciliation, service continuity, and reporting. Although web technology offers cross-platform accessibility, conventional web systems still depend heavily on internet connectivity or an Online-Only model. In this model, every transaction request must be transmitted directly to the server through an active connection. When the connection is interrupted or unstable, HTTP requests may fail and transaction payloads may be lost before they are securely stored. Operational observation of the Mangkasir cashier workflow indicated that this dependency produced three concrete failure risks: checkout requests could terminate before server acknowledgement, cashiers could remain blocked in a loading state, and unsaved payloads could require manual re-entry after connectivity returned. These risks are particularly consequential for small and medium retail settings where transaction volumes may fluctuate rapidly, network quality is not always stable, and cashier delays directly affect customer service continuity. For continuous transaction queues, network disruption is an architectural risk that affects business continuity and customer trust. Therefore, transactional systems require graceful degradation to remain functional, preserve critical data, and recover automatically when network infrastructure is compromised [4].

To address this problem, Progressive Web App (PWA) architecture combined with an Offline-First strategy offers a relevant solution. PWA technology provides a web-based application model with native-like capabilities, including installability, responsive interfaces, local caching, background processing, and reliability under unstable network conditions. Modern PWA development improves user experience by combining web accessibility with application-like behavior across devices [5]. PWAs are also defined as web applications that use modern browser capabilities to provide reliable, fast, and engaging experiences, including offline support and installability [6]. These capabilities are important in unstable connectivity environments because PWAs reduce dependence on continuous server access while maintaining interaction continuity [7]. For POS systems, however, the relevance of PWA is not limited to opening an application while offline. Its architectural value depends on whether browser-level components can preserve write operations safely, decouple user interaction from network availability, and support delayed synchronization without requiring cashiers to repeat transaction input.

The Offline-First strategy treats network failure not as an exception, but as a normal condition anticipated from the beginning of application design. Under this approach, the application remains operational locally when the server cannot be reached, while synchronization with the central database is postponed until connectivity is restored. Offline-First architecture is relevant for systems requiring efficient data handling and synchronization because it allows applications to accept user input during disruptions and process queued data after reconnection [8]. From a mobile and distributed systems perspective, Offline-First design improves usability and resilience by ensuring that core functions remain accessible under unstable or absent network conditions [9]. This model is suitable for transactional systems because it enables client-side request interception, local persistence, and synchronization scheduling. Compared with traditional online-only workflows, the method shifts temporary responsibility for transaction continuity from the server connection to the browser client, thereby allowing transaction entry to continue even when immediate server confirmation is unavailable.

However, existing implementations still reveal a research gap. Most PWA studies emphasize interface installability, static asset caching, and offline access for read-oriented content, such as news portals, tourism platforms, educational systems, or catalog-based applications. These systems mostly retrieve information rather than generate continuous transactional payloads. In contrast, write-heavy transactional systems require stronger guarantees because every user action creates data that must be preserved, ordered, transmitted, and validated accurately. The challenge is not only whether an application can be opened offline, but whether it can protect transaction queues, prevent duplication, maintain interface responsiveness, and ensure data consistency after reconnection. Accordingly, the novelty of this research is not the invention of Service Workers, IndexedDB, or Background Sync as individual technologies, because these are established browser mechanisms. The contribution lies in validating their combined architectural use for offline POS transaction queuing in the Mangkasir context, where repeated write operations, delayed synchronization, and controlled multi-device access must be handled as an integrated workflow.

Achieving this resilience requires autonomous client-side architectural components. Service Workers can operate as network proxies that intercept requests and manage caching or synchronization, while client-side databases can preserve transaction data before server transmission. IndexedDB is suitable because it provides an asynchronous local database mechanism capable of storing larger and more complex data than synchronous browser storage. IndexedDB has been discussed as a core technology for reliable offline applications because it enables structured local storage and supports asynchronous operations [10]. Studies on client-side processing also

emphasize the importance of browser-based storage for managing large data volumes in modern web applications [11]. Further validation is therefore needed to examine whether Service Worker, IndexedDB, and Background Sync can support transactional resilience in write-heavy systems. This architectural combination is expected to reduce immediate dependence on server availability by intercepting failed requests, persisting payloads locally, and executing queued synchronization once connectivity is restored.

Based on this gap, this research implements an Offline-First PWA architecture to strengthen transactional application resilience. The Mangkasir Point of Sale system was selected because it represents an intensive environment with rapid and continuous data mutations. In daily cashier operations, the system requires uninterrupted transaction input, fast response time, and reliable synchronization with the main server. Network failure may cause lost transaction queues, delayed sales recording, inconsistent database states, and reduced service quality. Mangkasir provides an empirical context for examining whether a web-based POS application can achieve resilience commonly associated with native or desktop-based systems. This study addresses PWA architecture design for a write-heavy system, the effectiveness of Service Worker and IndexedDB during connection loss, and the role of Background Sync in maintaining transaction data consistency. The main objective is to evaluate whether the proposed Offline-First PWA architecture can measurably preserve POS transaction queues locally, maintain user-interface continuity during network disruption, synchronize queued payloads after reconnection, and avoid duplicate entries during controlled multi-device synchronization testing.

II. LITERATURE REVIEW

Transactional computing systems, particularly Point of Sale (POS) platforms, are characterized as mission-critical, write-heavy applications that demand extremely high availability and near-zero downtime [2], [3]. These systems act as the operational center processing continuous financial queues [4]. Historically, achieving this level of operational stability required the utilization of native desktop or mobile architectures. Previous studies have demonstrated that native applications based on mobile and desktop programming languages provide robust reliability and data accuracy for micro, small, and medium enterprises transaction processing [5], [6]. However, native architectures fundamentally restrict cross-platform accessibility and incur high development and maintenance costs. On the other hand, while conventional cloud-based web architectures resolve the platform dependency issue, they introduce a fatal flaw for mission-critical and write-heavy systems: an absolute reliance on continuous internet connectivity, which leads to instantaneous data loss and operational paralysis during network disruptions [1]. From a resilience perspective, this limitation is not only a connectivity problem but also a transaction-state problem, because failed requests may create uncertainty over whether a payload has been created, transmitted, acknowledged, duplicated, or permanently lost.

To bridge the dichotomy between native stability and web accessibility, Progressive Web App (PWA) technology has emerged as an ideal paradigm. This technology integrates modern web application programming interfaces to deliver native-like experiences, network independence, and secure installability [7], [8]. Furthermore, it serves as a crucial technology to bridge the web accessibility gap in regions with limited network connectivity [9]. Despite its theoretical promise, a critical synthesis of recent literature reveals a systemic bias in current implementation trends. Existing PWA implementations should be differentiated into at least three categories: static offline access, low-frequency offline submission, and continuous offline transaction queuing. Static offline access mainly requires asset caching and page availability; low-frequency submission requires delayed form persistence; whereas continuous POS queuing requires ordered payload storage, retry control, duplicate prevention, and synchronization consistency. The majority of existing research limits the deployment to static, read-only systems or low-frequency transactional environments. For example, recent research successfully utilized this architecture to maintain offline access for a tourism village's web platform, but the system's core function was strictly limited to static information retrieval [10]. Similarly, other implementations focused on boarding house payment systems, which involve singular, monthly transactions rather than rapid, continuous data mutations [11]. These studies demonstrate the usefulness of PWA for availability and usability, but they do not fully test transaction intensity, concurrent queue accumulation, failure recovery during synchronization, or data-integrity validation under repeated write operations. This delineates a significant research gap, as the empirical validation of the architecture in continuous, write-heavy transactional environments remains largely unexplored.

Furthermore, the architectural approach to existing implementations often lacks true client-side autonomy. Previous development of web-based POS platforms for small businesses successfully achieved interface installability but relied on a monolithic architecture that remains heavily dependent on the server for business logic and data processing [12]. Such monolithic structures fail to provide resilient graceful degradation during offline states because the client cannot autonomously manage complex data inputs. In a write-heavy POS context, graceful degradation requires more than cached screens; it requires temporary client-side transaction ownership, deterministic queue ordering, idempotent submission keys, retry-safe synchronization, and mechanisms to minimize duplicate entries when the same logical transaction is resent. To achieve true resilience, the architecture must be decoupled, shifting the state management and data queuing entirely to the client side using modern frontend frameworks. Frameworks equipped with hybrid rendering capabilities, such as server-side and client-

side rendering, facilitate a decoupled infrastructure [13], [14]. This modern ecosystem enables seamless integration with advanced caching libraries and background synchronization mechanisms without requiring low-level browser scripting from scratch [15]. Nevertheless, the availability of these frameworks does not automatically establish transactional correctness; their effectiveness must be examined through controlled workload, recovery, and concurrency scenarios.

The realization of this resilient decoupled architecture relies on the Offline-First methodology, a paradigm shift that assumes the absence of a network connection as the default state of the application [16], [17]. In this framework, the autonomous network proxy intercepts requests and reroutes them to local storage when the network fails [13]. It also optimizes performance through advanced caching strategies, storing interface assets in cache storage to enable instant loading and reduce server request loads [18], [19]. Unlike traditional browser storage mechanisms, which are synchronous and severely limited in capacity, this methodology exploits the asynchronous, high-volume NoSQL object storage capabilities provided by local browser databases [20], [21]. The dynamic storage quota of this database is capable of handling the rapid insertion of serialized transaction payloads, commonly known as write-heavy queues, without freezing the browser's main thread or the user interface [22]. Subsequently, the background synchronization application programming interface defers the transmission of these queued payloads until stable connectivity is re-established, ensuring data integrity and consistency without requiring manual user intervention [16]. However, write-heavy resilience also raises unresolved concerns regarding eventual consistency, interrupted synchronization, partial server failure, local payload encryption, browser compatibility, and conflict resolution when multiple devices produce offline transactions concurrently.

In summary, while prior studies have validated the utility of modern web architectures for basic offline caching and native-like deployment, they have not adequately addressed the demands of write-heavy, mission-critical operations. More defensibly, limited empirical work has evaluated Service Worker interception, IndexedDB persistence, and Background Sync execution as an integrated Offline-First queueing architecture for POS transactions under controlled load and multi-device concurrency conditions. By synthesizing a decoupled frontend architecture with autonomous network interception and asynchronous local database queuing, this research uniquely positions itself to validate the Offline-First strategy as a robust replacement for native POS systems. Unlike previous studies that primarily demonstrate offline access or low-frequency submission, this research focuses on preserving continuous POS transaction queues, maintaining interface responsiveness, synchronizing delayed payloads after reconnection, and detecting duplicate-prevention behavior during concurrent synchronization. This approach aims to guarantee continuous operational resilience, prevent critical data loss, and maintain high availability amidst unpredictable and unstable network infrastructures in real-world scenarios.

III. METHODS

This research utilized a Research and Development (R&D) methodology combined with an applied qualitative approach to design, implement, and evaluate a Progressive Web App (PWA) architecture within an existing Point of Sale (POS) system. The R&D approach was selected because the study aimed not only to analyze an existing technical problem but also to produce and validate an architectural solution. Although qualitative data were used during the requirement identification stage, the evaluation process emphasized objective technical testing to validate the reliability of the Offline-First strategy in handling write-heavy transactional workloads.

A. Research Stages and Data Collection

The research was conducted through a structured sequence of stages to ensure systematic problem identification, prototype development, architectural integration, and system validation. These stages were arranged to guide the study from the initial analysis of the online-only POS system to the final evaluation of the Offline-First PWA implementation. The complete sequence of the research procedure is illustrated in Figure 1.

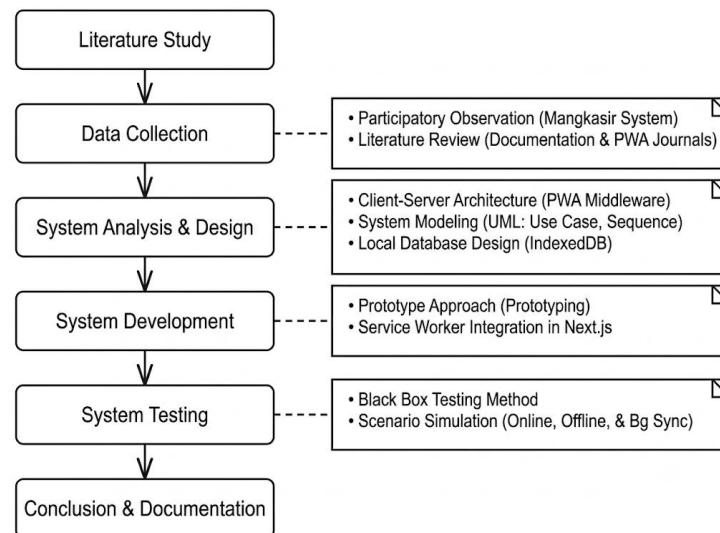


Fig. 1 Research Stages Diagram

Data collection involved participatory observation, in which the researcher acted as part of the Frontend Engineering team at PT Nugraha Kreasi Digital (Mangcoding). This role enabled the researcher to directly observe the architectural vulnerabilities of the existing online-only POS dashboard, especially during unstable network conditions. The observation data were supported by unstructured interviews with management to obtain technical permission for codebase modification and by a literature review on PWAs, Service Workers, IndexedDB, and Background Sync APIs.

B. System Development Method

The software development lifecycle adopted the Prototyping model because the proposed Offline-First mechanism needed to be tested incrementally before being integrated into the main system. This approach allowed the Service Worker, IndexedDB storage logic, and Background Sync mechanism to be constructed and evaluated in an isolated environment. By using this model, potential implementation errors could be identified early without disrupting the existing Next.js-based POS functionalities.

The development process consisted of several technical phases. First, the offline requirements of the core cashier transaction module were identified based on the failure points found in the existing online-only workflow. Second, an initial prototype was developed in a sandbox environment to test request interception, local data queuing, and synchronization behavior. Third, the validated modules were merged into the main architecture and subjected to final functional and non-functional testing.

C. System Analysis and Workflow Comparison

A workflow comparison was conducted to clarify the architectural differences between the existing system and the proposed Offline-First PWA solution. The purpose of this comparison was to identify how transaction payloads were handled before and after the implementation of the Service Worker and local database mechanism. The contrast between the current online-only workflow and the proposed Offline-First workflow is presented in Figure 2.

The pre-existing system relied entirely on Client-Side Rendering through the Next.js framework, where transaction payloads were sent directly to the server using HTTP requests. When the internet connection was interrupted, the fetch request failed and the transaction data could be lost before being stored securely. This condition created a critical operational vulnerability because cashier activities could not continue reliably during network disruptions.

The proposed architecture addressed this limitation by introducing a Service Worker as an autonomous middleware proxy between the user interface and the network layer. Under the new workflow, when a transaction is submitted during a network failure, the Service Worker intercepts the request and redirects the payload to local storage. The system then returns a mock success response to the user interface so that the cashier can continue working without application freezing. Once connectivity is restored, the Background Sync mechanism automatically transmits the queued transaction data to the main server.

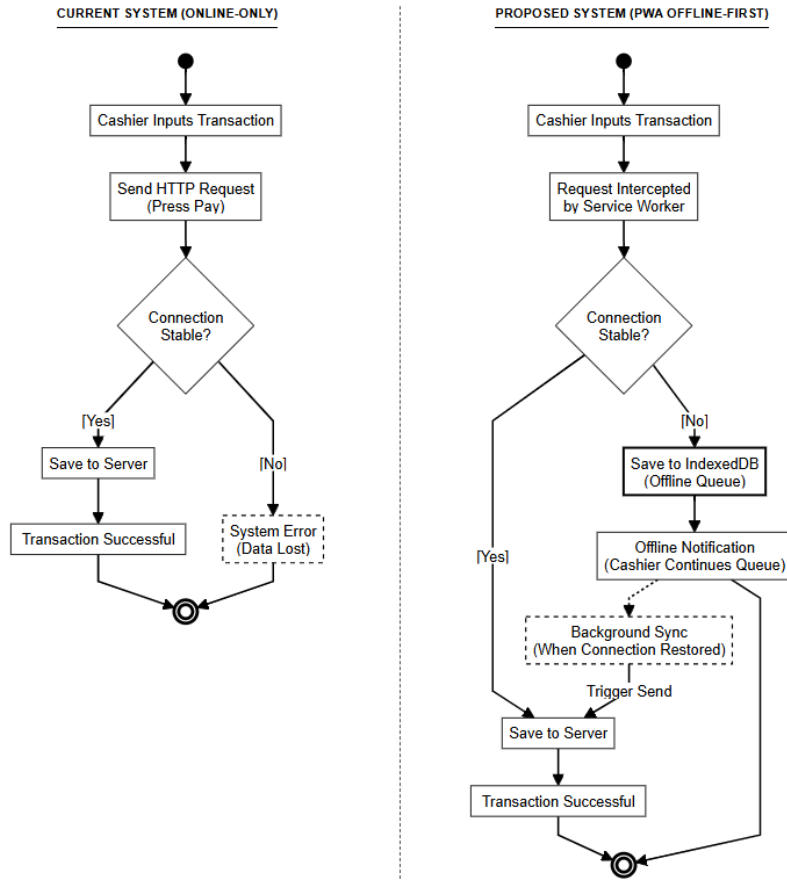


Fig. 2 Flowchart Comparison of Current and Proposed Systems Here

D. Local Database Design (IndexedDB)

A critical technical component of the Offline-First strategy was the design of client-side storage. Because the POS system is inherently write-heavy, the Indexed Database API (IndexedDB) was used as an asynchronous and object-oriented NoSQL database. This selection was intended to overcome the limitations of synchronous browser storage, such as Local Storage, which is less suitable for handling structured and high-volume transaction payloads.

A local database named `mangkasir_pwa_db` was initialized to store delayed transaction data during offline conditions. Within this database, a primary object store named `offline_transactions_queue` was designed to manage transaction payloads before synchronization. Each record contains a unique identifier, transaction payload, timestamp, and synchronization status. The structure of the IndexedDB object store used for offline transaction queuing is presented in Table 1.

TABLE 1
STRUCTURE OF THE OFFLINE_TRANSACTIONS_QUEUE OBJECT STORE IN INDEXEDDB

Field Name / Key	Data Type	Description	Attribute
id	String (UUID)	Unique identifier for each offline transaction to prevent data collision during synchronization.	Primary Key
transaction_payload	Object (JSON)	Main object containing shopping cart details and cashier session data.	Mandatory
timestamp	Integer	Transaction time in UNIX Epoch format for FIFO queue ordering.	Mandatory
sync_status	String	Data synchronization status (default: "pending").	Mandatory

E. System Testing Methods

System reliability was evaluated using functional and non-functional testing methods to ensure that the results were obtained objectively. These tests were designed to validate whether the proposed architecture could maintain transaction continuity, preserve data integrity, and recover automatically after network disruption. The testing procedures were divided into Black Box Testing and Performance and Load Sync Testing [23].

Functional testing was conducted using the Black Box Testing method to validate the application workflow across several simulated network environments [24]. The tested scenarios included stable online transactions, forced offline interception, Background Sync activation after network restoration, and low-bandwidth

simulation using Lie-Fi or Slow 3G conditions. This testing verified whether the system could respond correctly to each condition without requiring users to understand the internal program logic.

Non-functional testing was conducted through Performance and Load Sync Testing to evaluate scalability, synchronization speed, browser stability, and data integrity under transactional workloads [25]. The test simulated the accumulation of dummy transaction payloads in three volumes, namely 10, 50, and 100 transactions, to measure automatic synchronization execution time. In addition, concurrency testing was performed using three devices simultaneously to examine whether the backend server could process asynchronous transaction queues without data duplication, data loss, or race conditions.

IV. RESULTS

This section presents the empirical results obtained from the implementation and testing of the Progressive Web App (PWA) architecture with an Offline-First strategy in the Mangkasir POS system. The results include visual implementation outcomes, functional testing using the Black Box method, and non-functional testing related to load synchronization and multi-device concurrency. These findings are presented to demonstrate how the proposed architecture responds to network instability, preserves transaction data, and maintains system operability during offline and recovery conditions.

A. Visual Implementation Results

The implementation of the PWA architecture produced observable changes in the client interface during network transition states. When the browser detected a network disconnection, the system displayed a persistent offline indicator icon in the header and triggered a global toast notification informing the user that the connection had been lost. At the same time, the Service Worker intercepted failed HTTP requests and returned a mock success response to the interface, enabling the cashier to continue inputting transactions without causing the application to freeze. Figure 3 illustrates the adaptive user interface displayed during the offline state.

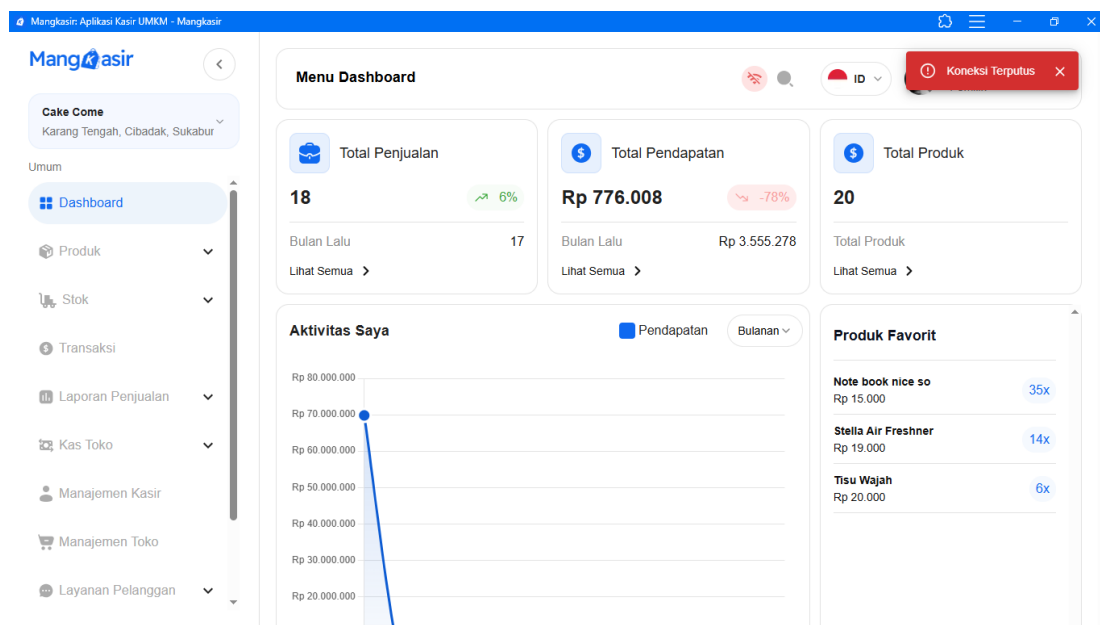


Fig. 3 Adaptive User Interface displaying offline indicator and toast notification.

Simultaneously, the Service Worker serialized the intercepted transaction payloads and stored them in the local IndexedDB database. The payloads were secured within the requests object store inside the servist-background-sync database, allowing transaction data to be temporarily retained on the client side before synchronization. This mechanism demonstrates that the system was able to preserve write-heavy transactional payloads locally when network access was unavailable. Figure 4 shows the representation of transaction queues stored within IndexedDB, including the request body and relevant structural metadata.

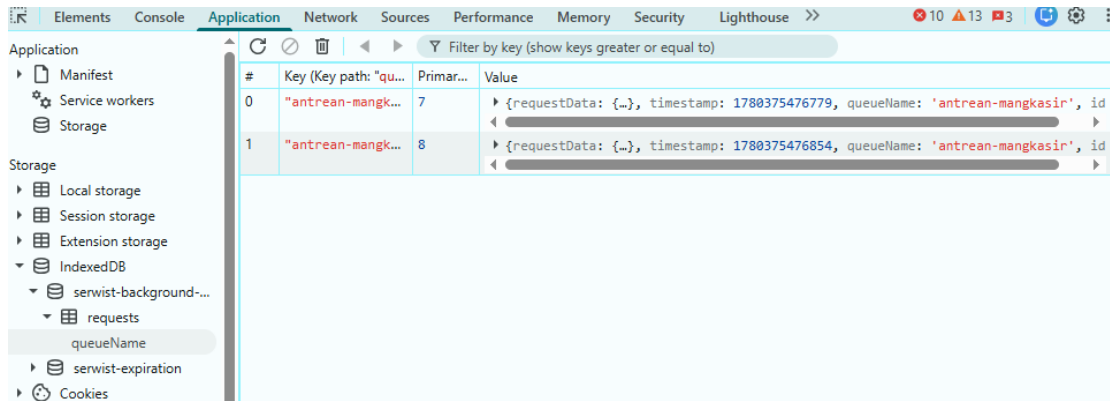


Fig. 4 Representation of transaction queues stored within IndexedDB

After network connectivity was restored, the Background Sync API executed the queued requests automatically in the background. The system then generated a native-like push notification to inform the user that the previously stored transactions had been successfully synchronized with the server. This feedback mechanism is important because it allows users to confirm that offline transactions have been processed without requiring manual re-submission. Figure 5 presents the push notification displayed after successful background synchronization.

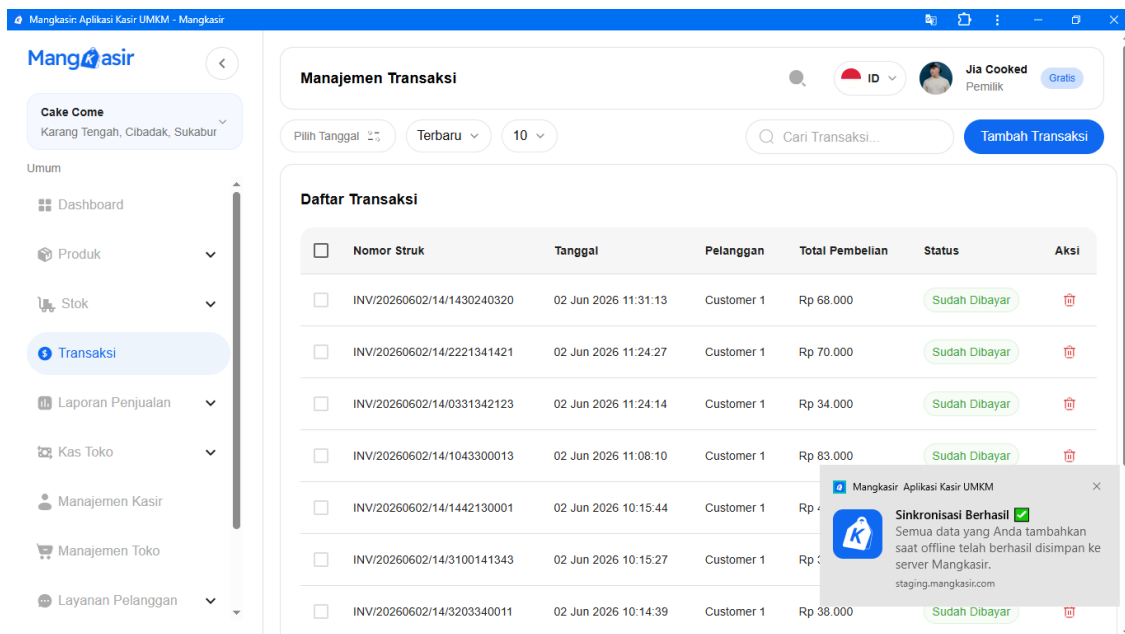


Fig. 5 Push notification confirming successful background synchronization

B. Functional Testing Results

Functional testing was conducted using the Black Box testing method to validate whether the application workflow operated correctly under different network conditions. The tested scenarios included normal online transactions, disconnected network interception, network recovery through Background Sync, and slow network conditions. The results indicate that the system performed according to the expected behavior in all scenarios. The detailed results of the functional testing are presented in Table 2.

TABLE 2
MANGKASIR SYSTEM FUNCTIONAL TESTING DETAIL (BLACK BOX)

Scenario	Expected Output	Actual Output	Status
Normal Network (Online)	Transactions are sent via HTTP protocol directly to the main server; UI shows success notification.	Request successfully reached the server endpoint (HTTP 200 OK); success notification appeared on UI.	Valid
Disconnected (Offline Interception)	Service Worker detects network absence, intercepts transaction, saves to IndexedDB, and UI gives mock success.	Transaction failed to reach network but was saved to sync-queue in IndexedDB; UI remained interactive and showed offline Toast Notification	Valid
Network Recovery (Background Sync)	OS and Service Worker detect connection recovery and execute data transmission in the background	Service Worker triggered sync event; IndexedDB data was emptied and synced to the main server without manual intervention	Valid

Slow Network (Lie-Fi / Slow 3G)	System limits timeout; if server response takes too long, transaction is routed to IndexedDB	After timeout, transaction was intercepted into the offline queue; UI was freed from the loading block	Valid
---------------------------------	--	--	-------

C. Non-Functional Testing Results

Non-functional testing focused on load synchronization to measure the execution time and stability of automatic data transmission from client-side local storage to the main server. The testing was conducted by simulating different volumes of dummy transaction payloads, namely low, medium, and high transaction loads. This test was necessary to determine whether the Background Sync mechanism could process increasing transaction volumes efficiently without disrupting browser stability. The recorded execution times are presented in Table 3

TABLE 3
RESULTS OF AUTOMATED BACKGROUND SYNCHRONIZATION LOAD TEST

Payload Volume	Total Execution Time (ms)	Average Time per Transaction (Seconds)
10 Transactions	10,006 ms	± 1.00 seconds
50 Transactions	30,456 ms	± 0.61 seconds
100 Transactions	42,597 ms	± 0.43 seconds

To evaluate the consistency of synchronization performance, a repeated testing scenario was conducted using a constant load of 10 transactions. This test was performed across three trials to observe whether execution time remained stable and whether data integrity was preserved across repeated synchronization attempts. The results show that all trials were completed successfully with no recorded transaction loss. The execution duration and data integrity status for each trial are documented in Table 4.

TABLE 4
AUTOMATIC SYNCHRONIZATION LOAD TEST RESULTS

Trial	Duration (ms)	Data Integrity Status
1	6,780 ms	Success (0% Loss)
2	6,757 ms	Success (0% Loss)
3	10,911 ms	Success (0% Loss)
Average	8,149 ms	100% Integrated

Finally, a concurrency test was conducted to validate data integrity when multiple devices accessed the system simultaneously using the same account session. Three devices, consisting of two smartphones and one laptop, were used to input 50 transactions each at the same time. This test was intended to examine whether concurrent transaction queues could be processed without data collision, duplicated entries, or race conditions. The results of the multi-device synchronization test are presented in Table 5.

TABLE 5
CONCURRENCY TESTING RESULTS (MULTI-DEVICE SYNC)

Test Parameter	Scenario Details	Actual Output	Status
Device Load	3 devices active simultaneously.	All three devices operated normally using a single account session simultaneously without interruption.	Valid
Simultaneous Transaction Load	Input of 50 transactions simultaneously from each device.	All requests from the three devices were successfully intercepted and executed by the server.	Valid
Final Data Consistency	Cross-validation between local data and the Main Database.	150 transaction data entries were recorded accurately with the correct timestamp integrity on the server.	Valid

V. DISCUSSION

The implementation of the Offline-First Progressive Web App architecture on the Mangkasir POS system demonstrates a feasible client-side resilience mechanism for preserving transaction queues during controlled network disruption scenarios. The integration of Service Workers as autonomous proxies and IndexedDB for asynchronous queuing ensured continuous operability without UI freezing. A highly significant finding from the load synchronization testing is the bulk processing efficiency achieved during automatic data transmission. The system processed 10 transactions in approximately 10.01 seconds, yet managed a tenfold increase (100 transactions) in only 42.60 seconds. This indicates that the Background Sync API handles bulk payload transmissions with increasing efficiency under the tested workload, minimizing the computational overhead typically associated with repetitive, synchronous HTTP requests. However, this result should be interpreted as evidence of architectural feasibility and synchronization behavior rather than proof of complete operational reliability across all production POS conditions. The evaluation confirms queue preservation, interface responsiveness, and delayed transmission, but it does not yet cover real payment settlement, inventory mutation

conflicts, server-side rollback, cashier correction after input error, or reconciliation after partial synchronization failure.

These empirical results offer a meaningful extension when compared to previous PWA research, which has predominantly favored static or low-frequency transactional models. For instance, while prior research successfully applied PWAs to maintain offline access for tourism information platforms, their architecture was strictly read-only and fundamentally incapable of safely processing dynamic user inputs during network failures [10]. Furthermore, previous developments of PWA-based POS platforms for small businesses achieved interface installability but relied heavily on monolithic, server-dependent structures that failed to provide graceful degradation, leaving the client vulnerable during offline states [12]. Similarly, earlier PWA implementations in boarding house payment systems focused only on singular, infrequent transactions rather than the rapid, continuous data mutations demanded by retail operations [11]. In sharp contrast, this study validates that a decoupled, client-side data queuing architecture can support continuous write-heavy operations under controlled network constraints. The contribution therefore lies not in the novelty of Service Worker, IndexedDB, or Background Sync as isolated technologies, but in evaluating their combined use as an integrated queue-preservation and synchronization pattern for POS transaction workflows.

The success of the multi-device concurrency testing, which processed 150 simultaneous transactions with zero observed data loss, suggests that the proposed web-based architecture may serve as a context-dependent alternative to native POS deployment in environments where connectivity disruption is frequent and cross-platform accessibility is important. By utilizing UUIDs and UNIX timestamp-based First In First Out (FIFO) sorting within the IndexedDB `offline_transactions_queue`, the system effectively reduced the likelihood of data collisions and race conditions during bulk synchronization to the main server. This indicates that shifting the state management entirely to the client side using modern hybrid rendering frameworks not only preserves operational continuity but also supports transactional ordering and duplicate-prevention behavior in the tested scenarios. Nevertheless, it would be premature to claim that this architecture is a direct cost-effective replacement for native POS applications, because no controlled comparison was conducted between native and PWA implementations under equivalent transaction volume, latency, device capability, security posture, offline duration, and recovery conditions. A more defensible interpretation is that Offline-First PWA design can reduce dependence on fully native architectures for selected cashier workflows, while still requiring additional validation before enterprise-wide substitution.

Despite these promising architectural advancements, this study possesses several technical limitations. The reliability of the Offline-First capability is heavily tethered to the client's browser engine support. While Chromium-based browsers natively execute the Background Sync API flawlessly, alternative engines—most notably WebKit used in iOS Safari—currently exhibit restricted support for background synchronization, often requiring the user to keep the browser tab active to trigger the queue transmission. This browser-engine dependency is a major deployment concern, not merely a technical inconvenience. A POS system that depends on browser-specific behavior cannot be considered broadly cross-platform until fallback synchronization strategies and compatibility tests are conducted across Chromium, Firefox, and WebKit environments. Additionally, the maximum storage capacity of IndexedDB is inherently bound by the dynamic hardware quota of the host device. Operating offline for extremely prolonged periods under massive, uninterrupted transactional loads could theoretically trigger out-of-memory errors on lower-end hardware. Such limitations indicate that Offline-First capability should be accompanied by storage monitoring, queue-size thresholds, user warnings, and administrative procedures for handling devices that remain offline beyond an acceptable operational window.

Data security also deserves central consideration because offline transaction queuing changes the threat model of the POS application. Transaction payloads may contain financial, product, cashier-session, and potentially customer-related information; therefore, storing these payloads in browser-managed local storage introduces risks that are less visible in online-only workflows. Device theft, shared cashier accounts, unauthorized browser inspection through developer tools, malicious scripts, and lack of encryption at rest may expose sensitive transactional data before synchronization occurs. In practical deployment, queue preservation should therefore be complemented by local payload encryption, strict session isolation, short-lived authentication tokens, role-based device access, integrity checks, and secure deletion after successful synchronization. Without these safeguards, the same mechanism that improves availability may increase exposure to local compromise. From a software engineering perspective, this finding implies that transactional resilience must be evaluated jointly with security resilience, because availability, integrity, and confidentiality are inseparable in financial application contexts.

Future research should specifically address these platform discrepancies by exploring fallback synchronization mechanisms, such as utilizing WebSockets or Server-Sent Events, to ensure seamless data transmission across iOS environments. Subsequent studies could also expand the local database schema to support complex, multi-node conflict resolution algorithms, which would allow for decentralized, collaborative transaction editing across multiple cashier branches before final synchronization. Finally, integrating advanced cryptographic standards within the IndexedDB storage should be investigated to elevate the security of the cached financial payloads against potential local exploitation. Further evaluation should also simulate duplicated retries, interrupted

synchronization, partial server failures, inventory update conflicts, transaction cancellation after offline entry, and post-sync reconciliation. These scenarios are essential for moving from architectural feasibility toward operational readiness, especially when POS systems must preserve not only local queues but also end-to-end business correctness across payment, inventory, reporting, and audit processes.

VI. CONCLUSIONS

This study implemented and evaluated an Offline-First Progressive Web App architecture for the Mangkasir POS prototype, demonstrating controlled evidence of local transaction queue preservation, continued cashier-interface responsiveness, and post-reconnection synchronization under simulated write-heavy conditions. By integrating a Service Worker for failed-request interception, IndexedDB for asynchronous local queuing, and Background Sync for deferred payload transmission, the prototype was able to preserve offline transaction data and synchronize queued requests without observed data loss or duplication in the tested scenarios. These findings contribute an applied architectural model for transactional resilience in web-based POS systems, particularly in environments where network instability threatens service continuity. However, the contribution should be interpreted as prototype-level resilience rather than evidence of full production or enterprise-scale reliability, because the evaluation did not yet include long-term field deployment, real payment-flow validation, server-side rollback behavior, security testing, or prolonged cashier operation across varied devices and network conditions.

Future research should extend this architecture through real-world cashier trials over multiple business days, repeated stress testing, and multi-browser compatibility assessment across Chromium, Firefox, and WebKit environments. Further development should also incorporate encrypted IndexedDB storage, automated storage-capacity management, idempotent server-side transaction handling, duplicate-retry control, and stronger synchronization recovery mechanisms for interrupted or partially completed uploads. To move from offline queue preservation toward full operational readiness, subsequent work must validate payment gateway reconciliation, offline inventory conflict resolution, transaction cancellation after offline entry, and integration with inventory, financial reporting, and audit modules. In this direction, Offline-First PWA architecture can become a stronger foundation for resilient transactional systems, provided that availability, data integrity, security, and business-process correctness are evaluated as interconnected requirements rather than isolated technical functions.

ACKNOWLEDGMENTS

The authors would like to express their deepest gratitude to PT Nugraha Kreasi Digital (Mangcoding) and the Mangkasir development team for granting observation permissions and providing codebase support throughout this research. Appreciation is also extended to the Faculty of Engineering, Computer, and Design at Universitas Nusa Putra for their comprehensive academic facilities and continuous support.

REFERENCES

- [1] A. K. Pattan and S. Gangishetti, "Progressive Fault Tolerance in API-First Front-End Applications: Architecture, Patterns, and Evaluation," *International Journal of Artificial Intelligence, Data Science, and Machine Learning*, vol. 5, pp. 253–261, 2024, doi: <https://doi.org/10.63282/3050-9262.IJAIDSML-V5I1P126>.
- [2] E. Techa Fattima, I. Aliffia Bingga, R. Anasthasia Seriulina, S. Rizkia Putri, T. Tamara, and V. Paramarta, "Analisis Transaction Processing System (TPS) dalam Konteks Pengembangan Sistem Informasi pada Organisasi," *COMSERVA : Jurnal Penelitian dan Pengabdian Masyarakat*, vol. 3, no. 10, pp. 4278–4289, Feb. 2024, doi: <https://doi.org/10.59141/comserva.v3i10.1217>.
- [3] M. Haubenschild and V. Leis, "Oltp in the cloud: architectures, tradeoffs, and cost," *The VLDB Journal*, vol. 34, no. 4, p. 42, Jul. 2025, doi: <https://doi.org/10.1007/s00778-025-00913-z>.
- [4] Aadesh Goraksha Landge, Amol Namdev Tupe, J.S.Ingale, Mahesh Haribhau Darekar, and Shubham Rajendra Gorade, "POSventory – An Integrated Inventory Management and Point of Sale Platform," *International Journal of Sciences and Innovation Engineering*, vol. 3, no. 4, pp. 1775–1783, Apr. 2026, doi: <https://doi.org/10.70849/ijsci03042614007>.
- [5] A. Suhargo, M. Marcelino, P. Vardi, S. Megawan, and D. Darwin, "Pengembangan Aplikasi Point of Sale (POS) berbasis Mobile untuk Membantu UMKM," *Jurnal Janitra Informatika dan Sistem Informasi*, vol. 5, no. 2, pp. 199–214, Oct. 2025, doi: <https://doi.org/10.59395/tt4rth66>.
- [6] A. Prastomo and S. Alfarisi, "Sistem Aplikasi Poin Of Sale (POS) Untuk Penjualan Toko Naila Cookies Berbasis Java," *Journal of Information System, Applied, Management, Accounting and Research*, vol. 8, no. 2, p. 456, May 2024, doi: <https://doi.org/10.52362/jisamar.v8i2.1501>.
- [7] B. R. Cherukuri, "Progressive web apps (PWAs): Enhancing user experience through modern web development," *International Journal of Science and Research*, vol. 13, no. 10, pp. 1550–1560, 2024, doi: <https://doi.org/10.21275/ms241022095359>.

- [8] Google Chrome Developers. (n.d.), "What are Progressive Web Apps?" Accessed: Apr. 09, 2026. [Online]. Available: <https://web.dev/explore/progressive-web-apps?hl=id>
- [9] B. R. Cherukuri, "Progressive Web Applications (PWA) : Combining the advantages of the web and native apps," *Int. J. Adv. Trends Comput. Sci. Eng.*, vol. 14, no. 4, pp. 205–208, Aug. 2025, doi: <https://doi.org/10.30534/ijatcse/2025/051442025>.
- [10] E. Hudianti and M. A. Nugroho, "Implementasi Progresive Web Apps untuk Sistem Pengelolaan Potensi Desa Wisata Kali Opak Tujuh Bulan," 2023. doi: <https://doi.org/10.24076/joism.2023v4i2.964>.
- [11] N. Dwiyanto and I. V. Paputungan, "Implementasi Teknologi Progressive Web App (PWA) Pada Sistem Informasi Kos-Kosan," *Journal of Information System Research (JOSH)*, vol. 6, no. 1, pp. 426–437, Oct. 2024, doi: <https://doi.org/10.47065/josh.v6i1.6074>.
- [12] I. D. Prayudha, M. A. Irwansyah, and H. Anra, "Rancang Bangun Aplikasi Point of Sale (POS App) Berbasis Progressive Web App untuk Usaha Mikro Kecil dan Menengah," *Jurnal Sistem dan Teknologi Informasi (JustIN)*, vol. 12, no. 2, p. 330, Apr. 2024, doi: <https://doi.org/10.26418/justin.v12i2.76824>.
- [13] S. Srivastava, H. Shukla, N. Landge, A. Srivastava, and D. Jindal, "A Comprehensive Review of Next.js Technology: Advancements, Features, and Applications," *SSRN Electronic Journal*, pp. 74–85, 2024, doi: <https://doi.org/10.2139/ssrn.4831070>.
- [14] Prof. G. Singh and N. I. Patel, "Server-Side Rendering (SSR) vs Client-Side Rendering (CSR) in React/Next.js: A Comparative Study on Performance, SEO, and Developer Experience," *International Journal Of Scientific Research In Engineering And Management*, vol. 10, no. 04, pp. 1–9, Apr. 2026, doi: <https://doi.org/10.55041/ijrsrem61203>.
- [15] Serwist Documentation, "Serwist: A collection of JavaScript libraries for Progressive Web Apps." Accessed: Apr. 09, 2026. [Online]. Available: <https://serwist.pages.dev/docs>.
- [16] K. Sirigiri, A. Narra, and A. Sirumalla, "Offline-first PWA : Case Study on Efficient Data Handling and Synchronization," *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, pp. 747–754, Jun. 2023, doi: <https://doi.org/10.32628/CSEIT25112782>.
- [17] S. H. Pothineni, "Offline-First Mobile Architecture: Enhancing Usability and Resilience in Mobile Systems," *Journal of Artificial Intelligence General science (JAIGS) ISSN:3006-4023*, vol. 7, no. 01, pp. 320–326, Dec. 2024, doi: <https://doi.org/10.60087/jaigs.v7i01.387>.
- [18] H. Muhamad and M. Taha, "Optimizing the Performance of Web Applications in Dynamic Network Environment: A Systematic and Comprehensive Analytical Survey," *Concurr. Comput.*, vol. 38, no. 6, Mar. 2026, doi: <https://doi.org/10.1002/cpe.70641>.
- [19] N. I. Mohammed, A. Jarde, G. Mackenzie, U. D' Alessandro, and D. Jeffries, "Deploying Machine Learning Models Using Progressive Web Applications: Implementation Using a Neural Network Prediction Model for Pneumonia Related Child Mortality in The Gambia," *Front. Public Health*, vol. 9, Feb. 2022, doi: <https://doi.org/10.3389/fpubh.2021.772620>.
- [20] V. K. Kotaru, "Introduction to IndexedDB," in *Building Offline Applications with Angular: Develop Reliable, Performant Web Applications for Desktop and Mobile Platforms*, Berkeley, CA: Apress, 2022, pp. 125–144. doi: https://doi.org/10.1007/978-1-4842-7930-4_7.
- [21] A. An, "Client-Side Processing and Storage of Large Data Volumes in Web Applications Using AI Techniques," *Computational Intelligence and Machine Learning*, vol. 6, 2025, doi: <https://doi.org/10.36647/ciml/06.01.a006>.
- [22] B. Jeong, S. Lee, and J. Park, "MIC: Memory analysis of IndexedDB data on Chromium-based applications," *Forensic Science International: Digital Investigation*, vol. 50, p. 301809, Oct. 2024, doi: <https://doi.org/10.1016/j.fsidi.2024.301809>.
- [23] R. A. Sianturi, A. M. Sinaga, Y. Pratama, H. Simatupang, J. Panjaitan, and S. Sihotang, "Perancangan Pengujian Fungsional Dan Non Fungsional Aplikasi Siappara Di Kabupaten Humbang Hasundutan," *Jurnal Komputer dan Informatika*, vol. 9, no. 2, pp. 133–141, Sep. 2021, doi: <https://doi.org/10.35508/jicon.v9i2.4706>.
- [24] A. Maspupah, "Literature Review: Advantages And Disadvantages Of Black Box And White Box Testing Methods," *Jurnal Techno Nusa Mandiri*, vol. 21, no. 2, pp. 151–162, Sep. 2024, doi: <https://doi.org/10.33480/techno.v21i2.5776>.
- [25] K. Sirigiri, A. Narra, and A. Sirumalla, "Offline-first PWA : Case Study on Efficient Data Handling and Synchronization," *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, pp. 747–754, Jun. 2023, doi: <https://doi.org/10.32628/CSEIT25112782>.